Task 03: Evaluate Operating-System Structures

Operating Systems (CSA04) — Assignment 1 (CO1)

Student Name: Rohit L
Register Number: 192511236
Course: Operating Systems (CSA04)
Marks: 4 Marks (Bloom's: K5 - Evaluate)
GitHub: `https://github.com/rohitaju`
Sub-directory: `Task03/`

> ***Task 3 Description:*** *Evaluate Monolithic, Layered, Microkernel, Modular (LKM), and Hybrid OS structures. Discuss their impact on reliability, performance, and updates. Defend structure choices for bedside monitors and the database server. Analyze alternative: Monolithic kernel for monitors.*

1. STRUCTURAL COMPARISON MATRIX

Architecture	Reliability & Fault Isolation	Raw Performance	Dynamic Updatability
Monolithic (Linux, BSD)	Low: All drivers, VFS, and network stacks share Ring 0. A single null pointer panics the whole OS.	Highest: Direct memory access and single-instruction traps without context switches.	Moderate: Can load modules (LKMs), but a faulty module crashes the running system.
Microkernel (QNX, seL4)	Extreme: Only IPC, scheduling, and basic memory primitives run in Ring 0. Drivers run in user space.	Lower: Message passing requires multiple user/kernel context switches and TLB invalidations.	Highest: Drivers and services can be stopped, updated, and restarted without a system reboot.
Modular (LKM) (Modern Linux)	Moderate: Modular structure improves organization, but modules execute in Ring 0 without memory barriers.	High: Near monolithic speeds once modules are bound into kernel space.	High: Allows dynamic loading/unloading of device drivers on active systems.
Hybrid (Windows NT, macOS)	Balanced: Microkernel-inspired design, but critical performance paths (GUI, network) run in Ring 0.	High: Avoids IPC penalties for common graphics and file operations.	Complex: Deep subsystem coupling makes independent service patching challenging.

2. DEFENDED STRUCTURAL SELECTIONS FOR HOSPITAL SYSTEMS

Bedside Patient Monitors: Microkernel Structure (e.g., QNX Neutrino). Bedside monitors require deterministic fault isolation. In a microkernel, device drivers (such as the pulse oximeter or ECG digitizer) run as user-space processes (Ring 3). If a driver experiences an illegal memory access or heap corruption, the microkernel detects the crash and restarts the driver in under 2ms, while the alarm monitor and display keep running.

Database Server: Modular Monolithic Structure (e.g., Enterprise Linux with LKMs). Database workloads are heavily I/O-bound. Running block drivers, the page cache, and network subsystems in Ring 0 eliminates IPC context-switching penalties, enabling maximum IOPS and transaction throughput.

3. ALTERNATIVE CONCEPT: MONOLITHIC KERNEL ON BEDSIDE MONITORS

- **Execution Cost (Time Complexity):** A monolithic system call requires ~100–300ns, while microkernel IPC requires ~1.5–3µs due to dual context switches. However, against a 10ms (10,000µs) alarm deadline, a 3µs IPC overhead consumes only 0.03% of the time budget, making the performance gain of a monolithic kernel negligible.
- **Failure Modes:** In a monolithic kernel, any faulty sensor driver executing an illegal memory write triggers an unrecoverable kernel panic, disabling the monitor and endangering the patient.
- **Memory Footprint (Space Complexity):** Microkernel images require only 1–4 MB RAM compared to 64–256 MB for a stripped monolithic Linux kernel, reducing hardware requirements and power consumption.